

# Adaptive Prefetching for Fine-grain Communication in PGAS Programs

Thomas B. Rolinger<sup>§</sup>  
 NVIDIA  
 Austin, TX USA  
 trolinger@nvidia.com

Alan Sussman  
 University of Maryland  
 College Park, MD USA  
 als@cs.umd.edu

**Abstract**—Applications that require distributed-memory systems and exhibit irregular memory access patterns present both productivity and performance challenges. This is largely due to the fine-grain communication that arises from irregular memory accesses to distributed data. The Partitioned Global Address Space (PGAS) model provides a globally shared address space and one-sided communication, making it well suited for implementing irregular codes. However, while the PGAS model provides productivity advantages, irregular applications have difficulty achieving high performance due to the cost of fine-grain remote accesses. One way to improve the performance of such PGAS programs is to prefetch remote data before it is needed, thereby hiding communication latency. The challenges of applying prefetching are computing the prefetch distance (i.e., how far ahead to issue prefetches), determining when prefetching will be profitable, and modifying the program to perform prefetching. In this work, we present an adaptive prefetching optimization that can be applied to PGAS programs with irregular memory access patterns. We target the Chapel parallel programming language, which implements a PGAS model. Our optimization leverages runtime information to adjust the prefetch distance and to pause/resume prefetching when that is likely to provide performance gains. Furthermore, we develop compiler support to automatically apply the optimization without requiring user intervention. We evaluate the optimization across five different distributed-memory systems and four workloads. We observe runtime speed-ups of 0.5 – 377x for the Chapel workloads when compared to not prefetching, and 3 – 215x when compared to implementations written in UPC, OpenSHMEM, and one-sided MPI.

**Keywords**—prefetching, PGAS, fine-grain communication, compiler optimization

## I. INTRODUCTION

The Partitioned Global Address Space (PGAS) model aims to make programming distributed-memory systems easier by providing a globally shared address space across the system, as well as mechanisms to perform fine-grain remote accesses [1]. Specifically, communication is performed in a one-sided manner, where a given process can simply `get` (read) or `put` (write) remote data. Languages and libraries that implement the PGAS model include Chapel [2], UPC [3], and OpenSHMEM [4]. Workloads that are characterized by irregular memory access patterns, such as graph analytics [5] and sparse matrix operations [6], are a natural fit for the PGAS

model. However, while the PGAS model provides productivity advantages for irregular codes, performance can suffer greatly due to fine-grain communication [7]–[10]. For languages like Chapel and UPC, each remote read issued by a process is a blocking operation, and as a result, processes can spend a significant amount of time waiting for data rather than performing computation.

One way to address the performance issue of fine-grain communication is to overlap the latency of remote accesses with other communication and/or computation. This can be achieved by *prefetching* the remote data before it is needed and placing it into local memory (i.e., cache). Prefetching can be performed in hardware [11]–[14] and software [15]–[17], where software prefetching is the focus of this work since it is more suitable for irregular memory access patterns.

We identify three challenges related to software prefetching for fine-grain communication: (1) computing the prefetch distance (i.e., how far ahead to issue prefetches), (2) determining when prefetching will be profitable, and (3) modifying the program to perform the prefetching optimization. Challenges (1) and (2) directly impact performance and are complicated by the fact that static decisions are inadequate at handling complex access patterns, programs with multiple phases, and different hardware systems. Challenge (3) can significantly impact developer productivity by requiring additional programmer effort to perform the optimization, which can cancel out the productivity advantages provided by the PGAS model.

In this paper, we present an adaptive prefetching optimization for PGAS programs that targets irregular memory access patterns within parallel loops, and that addresses all three challenges described above. Challenges (1) and (2) are addressed by leveraging runtime information to dynamically adapt (increase/decrease) the prefetch distance to ensure the prefetches are on time and to pause/resume prefetching when it is deemed advantageous for performance. By using runtime information, our prefetch optimization can be used on different systems, programs, and data without static tuning. Challenge (3) is handled by developing a compiler optimization that automatically identifies irregular memory access patterns to distributed data within parallel loops and then modifies the code to perform prefetching. In this work, we target Chapel programs, since Chapel is a modern PGAS language and provides a runtime managed cache for remote data that supports

<sup>§</sup>Work completed while employed at the Laboratory for Physical Sciences and a student at the University of Maryland.

prefetches [18].

Prior research has explored prefetching for remote data on distributed-memory systems and PGAS programs, but that work either only focuses on regular memory access patterns [19], [20], does not provide adaptive techniques to adjust prefetching behavior [21]–[23], or does not provide compiler support [18], [24]. Various forms of adaptive prefetching have been explored, but to the best of our knowledge only on shared-memory systems [13], [14], [25]–[28] and often requires new hardware support. In contrast, our work automatically applies adaptive prefetching to PGAS programs executing on distributed-memory systems and does not require new hardware designs.

The main contributions of our work are as follows:

- Design and implementation of an adaptive prefetching optimization for Chapel programs that uses runtime information to adjust prefetching behavior.
- Compiler support to automatically apply the prefetching optimization. Static analysis is developed to identify candidates for prefetching and code transformations are then applied to enact the optimization.
- Performance evaluation of the optimization across five different distributed-memory systems and four irregular workloads. We observe runtime speed-ups of 0.5 – 377x for the Chapel workloads when compared to not prefetching, and 3 – 215x when compared to implementations written in UPC, OpenSHMEM, and one-sided MPI.

The rest of this paper is organized as follows. Section II presents an overview of the Chapel programming language and other background information. Our adaptive prefetching optimization is described in Section III. A performance evaluation and accompanying results are presented in Section IV. Finally, Section V provides concluding remarks and describes future work.

## II. THE CHAPEL PROGRAMMING LANGUAGE

In this section we briefly describe the Chapel parallel programming language and its runtime managed cache for remote data. Chapel is a modern PGAS language and its remote cache provides a robust foundation for a prefetching optimization, as it already handles complicated issues like remote data coherency. Rather than providing a comprehensive overview of Chapel, this section will focus on the key features that are necessary to understand the details of our work.

### A. Terminology and Execution Model

A Chapel program is composed of *tasks* and *locales*. Tasks are defined as sets of computations that can be executed concurrently with respect to each other. Chapel employs a tasking layer that provides a thread implementation to execute tasks, where the default tasking layer is Qthreads [29]. A locale is a unit of machine resources (e.g., cores and memory) where tasks execute. Without loss of generality, throughout the rest of this paper we equate a task to a thread that is executing on a CPU core, and we assume a locale is a single compute node in a distributed-memory system.

Chapel uses fork-join parallelism where programs begin as one task executing on a single locale (i.e., a serial program). Additional parallel tasks can be created via language constructs that may or may not execute on other locales, but ultimately join back to a single task on one locale. This differs from other PGAS languages and libraries, like UPC and OpenSHMEM, that follow the Single Program Multiple Data (SPMD) model. In the SPMD model, multiple processes execute a copy of the same program, which means programmers include logic to correctly handle multiple processes executing each statement.

### B. Distributed Arrays

Lines 1–3 in Listing 1 show an example Chapel program that defines a 1-dimensional array `arr` of integers with indices 0 through 15 and then assigns a value to the array at index 4. Chapel refers to the index set of an array as its *domain*, which is a first-class object in the language (`dom` in Listing 1). Lines 5–6 create a block-distributed array `distArr` of integers with the same number of elements as `arr`, which is accomplished by passing in `dom` to the `createArray` constructor. A block distribution divides the array into contiguous chunks and assigns each chunk to a locale. As can be seen on line 6, the syntax to access a distributed array appears no different from accessing a non-distributed array (line 3). If the element to access is not mapped to the locale where the task is executing, Chapel will perform the necessary one-sided communication implicitly for the programmer. This provides a natural way of performing fine-grain remote accesses.

```
1 var dom = {0..15};
2 var arr : [dom] int;
3 arr[4] = 1; // local access
4
5 var distArr = Block.createArray(dom, int);
6 distArr[4] = 1; // may be remote access
```

Listing 1. Chapel arrays and distributions

### C. Data Parallel Loops

Chapel provides built-in syntax for parallel loops, where the data parallel `forall` loop is most relevant to our work. A `forall` loop divides the loop iteration space into chunks and then assigns each chunk to a task. The number of tasks created to execute a `forall` loop is determined by the number of cores available and the number of iterations. Listing 2 shows a `forall` loop that iterates over the domain of a distributed array to assign a value to each element. A crucial feature of `forall` loops is that they provide shared- and distributed-memory parallelism when iterating over distributed arrays/domains by executing iterations on the locale that an iteration is mapped to. As a result, the access on line 2 does not result in remote communication for accessing the array, since each iteration `i` is executed on the locale that “owns” `distArr[i]`.

```
1 forall i in distArr.domain {
2   distArr[i] = i;
3 }
```

Listing 2. Chapel forall loop

#### D. Remote Data Cache

Our adaptive prefetching optimization that will be described in Section III requires the existence of a cache that supports prefetching of remote data. Chapel provides a software write-back cache specifically for remote data and includes dirty bits, local memory consistency operations, and programmer-guided prefetches [18]. Each compute core on a locale has its own remote cache that operates independently from the other cores/caches. As a task in Chapel is ultimately mapped to a compute core on a locale, these caches can be considered to be per task. Each cache entry corresponds to a 1024 byte cache page, and each cache line is 64 bytes. Remote reads are automatically rounded up to fill cache lines and remote writes to nearby data will be aggregated, if possible. Additionally, remote memory access patterns that are sequential or have a predictable stride can be read ahead and cached. Chapel programs leverage the remote cache without any user intervention. However, Chapel provides a low-level user interface to perform prefetches for the remote cache, which issue non-blocking reads from specific remote addresses. These prefetches form the basis for our optimization, which will automatically insert prefetch instructions into a program.

### III. ADAPTIVE PREFETCHING OPTIMIZATION

Chapel's remote cache, as described in Section II-D, is effective at exploiting spatial and temporal locality in remote memory access patterns. The cache can provide a significant improvement for applications with dense/regular memory access patterns by reducing the amount of communication performed. However, the remote cache is ineffective at addressing the fine-grain communication that arises from irregular memory accesses to distributed data. One approach to improve the performance of codes with fine-grain communication is to *prefetch* remote data that will be accessed in later parts of the program, thereby overlapping the communication latency with computation or other communication. In this section, we present an adaptive prefetching optimization that automatically adjusts the behavior of the prefetches as the program is running, which can provide better performance than static approaches. Additionally, we develop compiler support that will automatically apply the optimization when it is valid to do so, requiring no user intervention.

#### A. High-level Description

The prefetching optimization targets memory access patterns of the form  $A[B[i]]$  within parallel `forall` loops, where  $A$  is a distributed array and  $i$  is the loop index variable. Such access patterns are common in scientific computing and graph analytics. More complicated variations of this access pattern are supported and will be discussed in Section III-D, but we refer to  $A[B[i]]$  for simplicity of presentation. The goal of the optimization is to insert the prefetch instruction `pf(A[B[i+dist]])` into the loop body, where `dist` is the prefetch distance. The `pf` instruction issues a non-blocking read for the specified address and places the data in the remote cache. In the case of  $A[B[i+dist]]$ , the ideal scenario is

to have the prefetched data available in the remote cache when iteration  $i+dist$  of the loop is executed. However, achieving that outcome depends largely on the choice of the prefetch distance `dist`, and where the prefetch instruction is placed within the loop body. The prefetch distance and placement of the prefetch instruction determine the amount of time between issuing the prefetch and needing to access the prefetched data.

#### B. Motivation for An Adaptive Approach

Simply inserting prefetch instructions with some predetermined prefetch distance is unlikely to provide good performance when considering different systems, workloads, and data that could be involved in any given program execution. Even if one is willing to statically tune the prefetch optimization for each system/workload/data combination, the memory access pattern can change throughout different phases of the program, further minimizing the effectiveness of a static approach. Additionally, each core on a system has its own remote cache that is independent from the other cores. As a result, prefetching needs to be considered per-task, where each task (i.e., core) has its own independent "stream" of memory accesses to consider, and therefore its own prefetching behavior. Distributed-memory systems can contain thousands of cores, resulting in an overwhelming number of prefetch streams to handle manually.

An alternative to static prefetching is an *adaptive* approach that can adjust prefetching behavior as the program runs. In this way, runtime-dependent information, such as network latency and memory access pattern characteristics, can be leveraged to inform the prefetching optimization. To provide an adaptive approach to prefetching, our optimization (1) gathers performance metrics at runtime that are specific to prefetching and the remote cache, and (2) modifies the `forall` loop to periodically stop a task's execution to adjust its prefetching behavior. For (1), we added new lightweight performance counters to Chapel's remote cache to keep track of prefetching metrics. For (2), once a given task has executed a specified number of loop iterations, which we will refer to as a *window*, the task will stop and execute a procedure that queries the remote cache metrics from (1) and then potentially adjust its prefetching behavior.

#### C. Adaptive Heuristic

There are three adjustments that can be made by the optimization to a given task's prefetching behavior: pause prefetching (i.e., do not issue any more prefetches), resume prefetching if it were paused previously, and increase/decrease the task's prefetch distance. Heirman et al. [25] use a similar heuristic for adjusting the prefetch distance, but we extend their work to also pause/resume prefetching when it can benefit performance. We developed a single heuristic that considers each of these behaviors, which is illustrated in Figure 1. Variables in all-caps (e.g., `PREFETCH`) correspond to constants, variables with subscripts (e.g.,  $T_{miss}$ ) are adjustable parameters to the heuristic, and all other variables (e.g., `dist`) are private counters/state kept by each task. Table I describes

TABLE I  
PARAMETERS FOR ADAPTIVE PREFETCHING

| Name              | Description                                                                                        | Value Used |
|-------------------|----------------------------------------------------------------------------------------------------|------------|
| $W_{size}$        | # of iterations to elapse before executing heuristic.                                              | 100        |
| $T_{late}$        | Upper bound on percentage of late prefetches allowed in a window before increasing distance.       | 20%        |
| $T_{unnecessary}$ | Upper bound on percentage of unnecessary prefetches allowed in a window before pausing prefetches. | 90%        |
| $T_{miss}$        | Upper bound on cache miss rate allowed in a window before resuming prefetches.                     | 95%        |
| $W_{late}$        | # of consecutive windows to be below $T_{late}$ before decreasing distance.                        | 8          |
| $W_{unnecessary}$ | # of consecutive windows to be above $T_{unnecessary}$ before pausing prefetches.                  | 4          |

each adjustable parameter used in the heuristic, as well as the values we use for our experiments in Section IV (Section IV-D details how we selected the values). We now describe the key features of the heuristic, including the adjustments it makes to prefetching behavior as well as the rationale behind the adjustment parameters.

1) *Window size*: When a loop is optimized for prefetching, each task’s `state` is initialized to `PREFETCH` and the prefetch distance `dist` is set to an initial value (e.g., 8). The choice of initial prefetch distance is not critical for the optimization since the distance can increase or decrease as the loop executes. The heuristic in Figure 1 is executed by each task after it has completed  $W_{size}$  iterations of the loop that is being optimized, where  $W_{size}$  defines the size of a window. This effectively determines the amount of runtime data that is gathered before the heuristic decides how to adjust a given task’s prefetch behavior. If  $W_{size}$  is too large, then the heuristic will not react fast enough to the changing behavior; if  $W_{size}$  is too small, then the heuristic will not have enough data to make informed decisions. Prior work that focus on prefetching with hardware support often use clock cycles, instructions, or time as a means to define a window, where a thread can be interrupted by the hardware after a specified number of cycles/instructions/milliseconds [13], [25], [27]. However, our approach operates entirely in software, so we rely on loop iterations to determine the window size.

2) *Pausing Prefetches*: Performance gains are not always possible via prefetching, as certain types of remote memory access patterns will benefit more than others from prefetching. Specifically, memory access patterns that are sequential or have a predictable stride are likely to be read ahead and cached by Chapel’s remote cache. In such cases, the prefetching optimization would issue *unnecessary* prefetches to the remote cache, which are prefetches for data that is already in the cache. A high rate of unnecessary prefetches is indicative

of regular/dense memory access patterns that exhibit spatial and temporal locality, which preclude explicit prefetches. While there is no additional communication overhead when issuing an unnecessary prefetch, there is still overhead in “preparing” for a prefetch that will not end up being issued, and an excessive rate of unnecessary prefetches can lead to performance loss.

In Figure 1, if a given task’s `state` is set to `PREFETCH`, then the heuristic will check whether the task should pause its prefetches based on the rate of unnecessary prefetches observed in previous windows. The threshold for too many unnecessary prefetches,  $T_{unnecessary}$ , is a percentage and should be set fairly high (e.g., 90%). Furthermore,  $T_{unnecessary}$  needs to be exceeded for a specified number of consecutive windows ( $W_{unnecessary}$ ). The rationale behind these parameters is that the overhead of issuing unnecessary prefetches is not significant enough that the optimization should pause prefetches if only a few unnecessary ones are detected. As pausing prefetches effectively turns off the optimization, it should only be done when large performance losses are likely to occur if prefetches continue to be issued.

3) *Resuming Prefetches*: If prefetches have been paused due to too many unnecessary prefetches, it may be beneficial to resume the prefetches at some point. This is likely when the memory access pattern undergoes different phases throughout a kernel/loop. For example, a sparse matrix or graph can contain different underlying structures, where some structures are dense while others are sparse. If the prefetching heuristic were to pause prefetches when regular memory access patterns are observed but never attempt to resume, then large performance gains may be missed when executing the irregular memory access patterns.

In Figure 1, if a given task’s `state` is set to `NO_PREFETCH`, then the heuristic will check whether prefetches should resume based on the observed miss rate in the remote cache. A high miss rate is indicative of irregular memory access patterns that could benefit from prefetching. If the miss rate exceeds  $T_{miss}$ , then `state` will be set to `PREFETCH` and prefetches will resume immediately with the last prefetch distance used before prefetches were paused.  $T_{miss}$  should be set high (e.g., 95%), as prefetches should only resume if the memory access pattern is highly irregular. The heuristic does not require that  $T_{miss}$  be exceeded for some number of consecutive windows. Unlike unnecessary prefetches, which can be tolerated to some degree, excessive remote cache misses lead to increased communication latency and should be addressed quickly.

4) *Prefetch Distance Adjustment*: An important factor that must be considered when issuing prefetches is the prefetch distance. The choice of prefetch distance depends on the network latency of the system, the execution time of a loop iteration, where the prefetch instruction is placed in the loop body, and the memory access pattern being prefetched. Additionally, the “best” distance to use for a given program may change throughout its execution. Such challenges complicate the use of a static prefetch distance chosen at compile time. To address this challenge, the optimization will increase/decrease a given

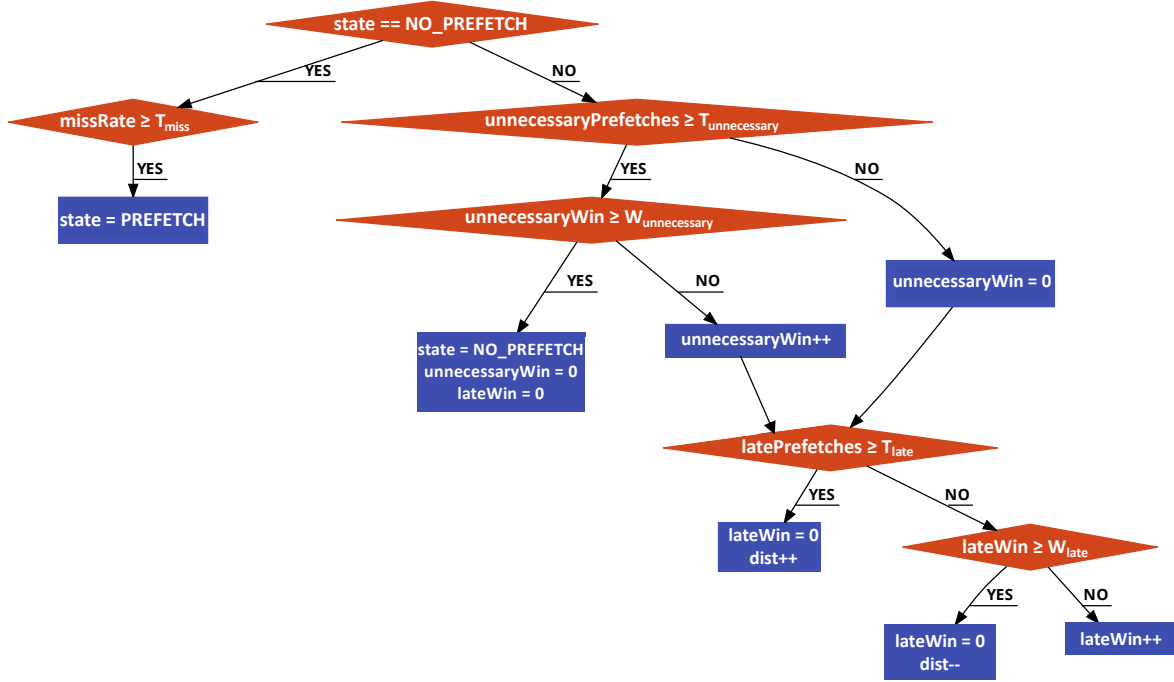


Fig. 1. Flow chart for the adaptive prefetching heuristic that is executed by each task after every  $W_{size}$  iterations of the loop being optimized. Diamonds represent decisions based on parameters defined in Table I and rectangles represent executed statements. Variables in all-caps correspond to constants, variables with subscripts are adjustable parameters to the heuristic, and all other variables are private counters/state kept by each task.

task's prefetch distance at runtime based on the timeliness of the prefetches issued within a window. We model this distance adjustment approach after the work by Heirman et al. [25], but we apply it to remote memory accesses in the PGAS model. Specifically, the optimization considers how many *late* prefetches occurred in a window, where a prefetch is late if the prefetched data has not arrived by the time a task needs to access it. Such prefetches are not entirely bad, as they may still be hiding part of the communication latency. Furthermore, a small increase to the prefetch distance is likely to improve performance by providing more time for subsequent prefetches to arrive.

In Figure 1, assuming that a given task's *state* is set to *PREFETCH* and there are not too many unnecessary prefetches, the heuristic will determine whether the prefetch distance *dist* should be increased, decreased, or remain the same. If the percentage of late prefetches in the window exceeds  $T_{late}$ , then *dist* is increased by one. Otherwise, if the task's percentage of late prefetches has been below  $T_{late}$  for  $W_{late}$  consecutive windows, then *dist* is decreased by one to avoid *early* prefetches, which happen when prefetched data is evicted before being accessed. Because late prefetches lead to larger performance losses than unnecessary prefetches,  $T_{late}$  should be set relatively low (e.g., 10-20%) so that the heuristic can more quickly address them. The purpose of  $W_{late}$  is to avoid early prefetches, which can occur when the prefetch distance is too large. By increasing/decreasing the prefetch distance under these conditions, the heuristic attempts to have

prefetches arrive close to just in time.

#### D. Compiler Support

The PGAS model aims to make writing distributed-memory programs with irregular memory access patterns easier by providing a global address space and one-sided communication. However, to achieve good performance for PGAS programs, programmers are often required to manually apply optimizations, such as prefetching, which can negate the productivity advantages provided by the PGAS model. We now discuss how to bridge the gap between performance and productivity by automatically applying our adaptive prefetching optimization to Chapel programs through the compiler using static analysis and code transformations. As a result, the compiler optimization operates by simply turning on a compiler flag, with no further intervention required by the user. We build upon our prior compiler optimization for selective data replication [9] and integrate both optimizations within a single framework rather than as separate passes. We briefly describe how both optimizations interact with each other within the compiler.

1) *Static Analysis*: The compiler optimization begins by using static analysis to identify memory access patterns of the form  $A[B[i]]$  within *forall* loops, where  $A$  is a distributed array. More complicated forms are supported, such as those that consist of arithmetic expressions and multiple variables (e.g.,  $A[B[i*2]+j]$ ). These types of access patterns are likely to result in fine-grain communication. A majority of the static analysis performed for the prefetching optimization

involves ensuring that the loop containing the prefetch candidate is valid. In this case, valid means that the compiler can statically reason about how the loop iterations progress. This is important for prefetching because the optimization needs to ensure that prefetches will not be issued beyond the ends of the array. A valid loop is one that directly iterates over an array's elements or its domain, or iterates over an explicit range (e.g., `forall i in 0..4`), and has a stride that can be determined before the loop executes. From these loops, the optimization can determine the first and last value of the loop index, as well as the number of iterations. Additionally, we allow for prefetch candidates to be within `for` loops that are nested within `forall` loops, where the same static analysis is applied to the `for` loop.

Another aspect of the static analysis is ensuring that the loop index variable is used within the prefetch candidate's array access expression. As the prefetching performed by our optimization focuses on array accesses in loops and attempts to look ahead some number of loop iterations, if the candidate access does not use the loop index variable within its access computation, then it cannot be prefetched by the optimization. Furthermore, static analysis must ensure that any other variables used within the candidate's access computation can be reasoned about w.r.t. how the loop progresses. To this end, such variables are required to be read-only within the scope of the loop body, which greatly simplifies the analysis that is required to ensure the prefetches are issued to valid array locations. We developed interprocedural and alias analyses to ensure the variables are read-only within any procedures that are called from the loop body, and to track references/pointers derived from the variables.

Last, if the data to be prefetched is a record (i.e., struct), static analysis will identify which fields of the record are accessed in the loop and issue prefetches specifically to those fields. Issuing a single prefetch to encapsulate multiple fields could lead to unnecessary data movement if the fields are not adjacent to each other in the record. We also use interprocedural analysis to identify field accesses that are issued within procedures that are called from the loop body.

2) *Code Transformations*: If static analysis determines that a given prefetch candidate and loop can be optimized, then the compiler will apply a series of code transformations to enact the prefetch optimization. First, procedure calls are inserted before the loop to compute frequently used constants, such as the bounds and stride of the loop. Next, per-task private variables are generated to keep track of the internal state of the adaptive prefetching heuristic state shown in Figure 1 (e.g., `dist` and `state`). The body of the loop is then modified around the prefetch candidate access. Specifically, an if-statement is inserted before the candidate to check whether  $W_{size}$  loop iterations have elapsed, which will then trigger the adaptive heuristic. Next, the prefetch instruction is generated and the address to prefetch is computed using the loop index variable, the prefetch distance, and the stride of the loop. If the candidate access is  $A[B[i]]$ , where  $i$  is the loop index variable and `strd` is the stride of the loop, then the compiler

will generate the expression  $A[B[i + (dist * strd)]]$  for the prefetch instruction. Last, a bounds check is inserted around the prefetch instruction to ensure that the generated prefetch address will not be out of bounds. Rather than using Chapel's built-in support to do array bounds checking, which can be expensive, we simply check whether the "future" value of the loop index variable is within the bounds of the loop (i.e., it is a value that would be yielded by the loop in some future iteration).

3) *Integration with Selective Data Replication*: In prior work, we designed and implemented a compiler optimization to perform *selective data replication* (SDR) [9]. SDR uses the inspector-executor technique [30]–[32], where the *inspector* determines which elements of a distributed array within a `forall` loop will be accessed remotely from each locale and the *executor* replicates the remote elements on the corresponding locale and redirects the remote accesses within the loop to the local copies. SDR also targets irregular array accesses of the form  $A[B[i]]$ , but can be considered a more powerful optimization than adaptive remote prefetching, as well as a more restrictive optimization. The prefetching optimization presented in this paper aims to overlap the latency of remote accesses in `forall` loops with other communication and/or computation. On the other hand, SDR completely eliminates the remote accesses in the loop by making explicit copies of the remote data prior to the loop. However, SDR can only be applied if (1) the cost of the inspector/replication can be amortized over multiple executions of the executor (i.e., `forall` loop), and (2) the data being replicated is read-only throughout the loop or can be formulated as a reduction operation. More details of SDR are provided in our prior work [9].

Rather than treat SDR and adaptive prefetching as two distinct optimization passes in the compiler, we combine their static analyses and code transformations. By doing so, users still only need to enable a single compiler flag to make use of both optimizations, and users do not need to make the decision about which optimization should be applied in any given scenario. Instead, our static analyses use the rules described above and in our prior work to determine which optimization can and should be applied to ensure that program correctness is maintained and that performance improvements are likely to be achieved. If the conditions for applying SDR can be met, then the compiler will give priority to SDR over prefetching because SDR is likely to provide larger performance gains. However, because applying SDR has many restrictions, the compiler will employ the prefetching optimization when SDR cannot be applied. Some of these decisions must be made at runtime, as some information cannot be inferred until the program executes. In such a case, the compiler will perform the code transformations for both optimizations, but also generate conditional logic that is evaluated at runtime to perform the optimization that is determined to be applicable both for program correctness and to maximize performance.

TABLE II  
SYSTEMS USED IN PERFORMANCE EVALUATION

| Name    | CPUs               | Interconnect | Topology      |
|---------|--------------------|--------------|---------------|
| FDR-56  | Intel Xeon E5-2650 | 56 Gb/s IB   | Single Switch |
| HDR-100 | AMD EPYC 7763      | 100 Gb/s IB  | Fat-tree      |
| HDR-200 | AMD EPYC 7713      | 200 Gb/s IB  | Fat-tree      |
| Cray XC | Intel Xeon E5-2699 | Aries        | Dragonfly     |
| Cray EX | AMD EPYC 7662      | Slingshot-11 | Dragonfly     |

#### IV. PERFORMANCE EVALUATION

In this section we present a performance evaluation of our adaptive prefetching optimization across five different distributed-memory systems and four irregular workloads written in Chapel. The motivation for considering multiple systems, workloads, and data sets is to demonstrate the effectiveness of the optimization at adapting prefetching behavior in response to different hardware parameters (e.g., network latency and topology) and different types of memory access patterns. In summary, we conducted over 1,300 experiments when factoring in the different systems, workloads, data sets, and number of nodes utilized. We observe that performance gains are achieved in 75% of those 1,300 experiments. Furthermore, these performance gains are achieved without modifying the original codes. Instead, the optimization is applied automatically via the compiler, as described in Section III-D. Therefore, our results demonstrate that we can achieve good performance without negatively impacting developer productivity.

##### A. Experimental Setup

Table II presents the five distributed-memory systems that we utilize in our experiments, where “IB” refers to Infiniband, and “FDR” and “HDR” refer to the data rates of the Infiniband networks. On each system, we compile our Chapel workloads using the same version of Chapel (1.31.0), which is modified to perform the adaptive prefetching optimization. We use GASNet [33] as the communication layer on all systems since it supports non-blocking remote reads, which are necessary for prefetching remote data. The Infiniband systems use the `ibv` communication substrate, while the Cray XC and EX systems uses the `aries` and `ofi` substrates, respectively. For each workload and data set, we run five trials and report the average result. We observed that the variation between trials did not exceed 5%. The primary performance metric reported is the runtime speed-up achieved when prefetching is applied versus the same workload executing without prefetching (referred to as *no-prefetching* throughout this section). Experiments are executed on up to 64 nodes, except for the FDR-56 system which only has 16 nodes. We equate an entire compute node to a single Chapel locale and will use the terms interchangeably.

##### B. Workloads

The workloads we evaluate are parallel implementations of IndexGather (IG), Sparse Matrix-Vector Multiply (SpMV), PageRank (PR), and Single Source Shortest Path (SSSP). IG

is taken from the Bale kernels<sup>1</sup>, which are designed to stress internode irregular communication patterns. Specifically, IG performs uniformly random reads to a distributed table/array. SpMV is a fundamental operation in sparse linear algebra and graph analytics, and can be thought of as an application of the IG communication pattern. However, SpMV operates on sparse matrices, which can exhibit a variety of different sparsity structures that lead to different memory access patterns. PR is a graph analytic that computes a ranking/importance value for each vertex in the graph and relies on performing several SpMV-like operations until a convergence threshold is reached. Finally, SSSP is a graph analytic that finds the shortest path (i.e., minimum edge weight) from a specified root vertex to all other reachable vertices in a graph. We use the delta-stepping algorithm in our implementation of SSSP [34]. The common theme among all four workloads is irregular memory accesses to distributed arrays, which are of the form  $A[B[i]]$  and can be prefetched via the optimization described in Section III. These access patterns are nested in parallel `forall` loops and contribute to a majority of the total runtime for the workload because they result in fine-grain communication. Note that the selective data replication optimization from our prior work [9] and described briefly in Section III-D3 can only be applied to the PR workload.

##### C. Data Sets

In our evaluation, we consider a mix of synthetic and real-world data sets when evaluating the workloads. Following the Bale specification for IG, we generate uniformly random values to use as indices into a distributed array/table with  $10^5$  elements. We then vary the total number of reads issued to the table from 100 million to 800 million. Table III presents the data sets used in the remaining workloads, which are sparse matrices/graphs. For SpMV, we consider 35 sparse matrices from the SuiteSparse Matrix Collection [35], which are drawn from domains such as graph analytics, computational fluid dynamics, and finite element analysis. We also include two sparse matrices from the NAS Parallel Benchmarks [36] specification for Conjugate Gradient (NAS\_E and NAS\_F). For PR, we select three data sets from the 35 SuiteSparse matrices that represent graphs: arabic-2005, sk-2005, and MOLIERE\_2016. We also evaluate a synthetic graph generated according to the Graph500 benchmark specification (s-28). Graph500 graphs are characteristic of graphs whose degree distributions follow a power law [5]. For SSSP, we focus our evaluation on Graph500 graphs s-26, s-27, and s-28. Table III describes all of these matrices/graphs in order of decreasing number of non-zero/edges.

##### D. Prefetch Parameter Selection

The adaptive prefetching optimization presented in Section III has six parameters that need to be set prior to running a program. These parameters are described in Table I and determine important factors such as when to pause/resume

<sup>1</sup><https://github.com/jdevinney/bale>

TABLE III  
GRAPHS/MATRICES USED IN EXPERIMENTS

| Name              | Rows | Non-zeros | Density (%) |
|-------------------|------|-----------|-------------|
| NAS_F             | 54M  | 55B       | 0.002       |
| AGATHA_2015       | 184M | 11.6B     | 3.4e-5      |
| s-28              | 268M | 8.5B      | 1.2e-5      |
| MOLIERE_2016      | 30M  | 6.7B      | 7.3e-4      |
| NAS_E             | 9M   | 6.6B      | 0.008       |
| GAP-urand         | 134M | 4.3B      | 2.4e-5      |
| GAP-kron          | 134M | 4.2B      | 2.3e-5      |
| s-27              | 134M | 4B        | 2.4e-5      |
| com-Friendster    | 66M  | 3.6B      | 8.4e-5      |
| mycielskian20     | 786K | 2.7B      | 4.4e-1      |
| s-26              | 67M  | 2B        | 4.8e-5      |
| sk-2005           | 51M  | 1.9B      | 7.6e-5      |
| it-2004           | 41M  | 1.2B      | 6.7e-5      |
| webbase-2001      | 118M | 1B        | 7.3e-6      |
| uk-2005           | 40M  | 936M      | 6.0e-5      |
| nlpkt240          | 28M  | 803M      | 1.0e-4      |
| arabic-2005       | 23M  | 640M      | 1.2e-4      |
| mawi_201512020330 | 226M | 480M      | 9.4e-7      |
| kmer_V1r          | 214M | 465M      | 1.0e-6      |
| nlpkt200          | 16M  | 465M      | 1.8e-4      |
| kmer_A2a          | 171M | 361M      | 1.2e-6      |
| stokes            | 11M  | 349M      | 2.7e-4      |
| Queen_4147        | 4M   | 334M      | 1.9e-3      |
| mycielskian18     | 197K | 301M      | 7.8e-1      |
| uk-2002           | 19M  | 298M      | 8.7e-5      |
| HV15R             | 2M   | 283M      | 7.0e-3      |
| mawi_201512020130 | 129M | 270M      | 1.6e-6      |
| rgg_n_2_24_s0     | 17M  | 265M      | 9.4e-5      |
| com-Orkut         | 3M   | 234M      | 2.5e-3      |
| indochina-2004    | 7M   | 194M      | 3.5e-4      |
| kron_g500-logn21  | 2M   | 182M      | 4.1e-3      |
| mawi_201512020030 | 69M  | 143M      | 3.0e-6      |
| vas_stokes_4M     | 4M   | 132M      | 6.9e-4      |
| Bump_2911         | 3M   | 131M      | 1.5e-3      |
| Cube_Coup_dt0     | 2M   | 129M      | 2.8e-3      |
| rgg_n_2_23_s0     | 8M   | 127M      | 1.8e-4      |
| Flan_1565         | 1.6M | 119M      | 4.9e-3      |
| hollywood-2009    | 1.1M | 115M      | 8.9e-3      |
| ML_Geer           | 1.5M | 111M      | 4.9e-3      |
| delaunay_n24      | 17M  | 101M      | 3.6e-5      |

prefetches. For the experiments presented in this section, we performed a large parameter sweep to test different combinations of values. Specifically, we evaluated 20,000 different combinations across the HDR-200 and Cray XC systems for the four workloads on two data sets per workload when using 8 locales. We then used the condorcet method<sup>2</sup> to rank all of the combinations based on their runtimes. The values in the rightmost column in Table I represent the top ranked configuration.

It is important to note that some parameter configurations are equivalent in their underlying behavior due to correlations between the parameters. For example, when  $W_{size}$  is 100 and  $W_{late}$  is 8, then 800 iterations must elapse where the number of late prefetches is below  $T_{late}$  before we attempt to decrease the prefetch distance. This behavior also occurs when  $W_{size}$  is 50 and  $W_{late}$  is 16. Using this fact, we can remove redundant configurations from the parameter sweep. Additionally, we can use the rationale described in Section III-C to limit the range of values we test. For example, we do not need to

<sup>2</sup><https://pypi.org/project/condorcet/>

consider small values for  $T_{miss}$  or  $T_{unnecessary}$ , as they both should only trigger the heuristic in the extreme case when the memory access pattern is almost entirely sequential or irregular, respectively.

Additionally, the parameter configuration we chose is not the only one that provides a performance benefit over no-prefetching. Specifically, more than 60% of the configurations that were tested perform better than no-prefetching. Furthermore, we use the same values from Table I on all the systems, workloads, data sets, and number of locales evaluated without performing additional parameter searching. As a result, 99% of the more than 1,300 experiments we perform for our evaluation were not tuned in any way w.r.t. the prefetching parameters. However, as will be presented in our results below, we still achieve performance gains across the different systems and workloads.

### E. System Adaptability Results

Table IV presents the average speed-ups achieved on each system from Table II across the four workloads. These speed-ups are relative to the no-prefetching baseline code, and are averaged across all data sets and locale counts. These results demonstrate the optimization's ability to adapt to different system configurations, as will be discussed in detail below.

1) *IG*: For IG, the achieved speed-ups are significant on the Infiniband systems (FDR-56, HDR-100, and HDR-200) due to the highly irregular memory access patterns and the fact that Infiniband networks are not well optimized for fine-grain communication. As a result, the prefetching optimization is able to reduce runtimes from tens of minutes to less than 5 seconds by hiding the communication latency and making better use of the remote cache. Note that the FDR-56 system's interconnect bandwidth is much smaller than the other IB systems (56 Gb/s versus 100 Gb/s and 200 Gb/s), which makes it more sensitive to the number of in-flight prefetches it can support before oversaturating the network. This leads to smaller relative speed-ups. On the other hand, the Cray XC achieves much lower speed-ups because of the Aries interconnect, which is better optimized for fine-grain communication than Infiniband. Last, the Cray EX system with the Slingshot-11 (SS-11) interconnect achieves the highest relative speed-ups w.r.t. the other systems, with a max speed-up of 377x. While SS-11 is optimized for small messages, the GASNet communication layer is not well-optimized for SS-11 at this time. However, GASNet provides the non-blocking reads that are necessary for prefetching, while the OpenFabrics `ofi` communication layer does not. Therefore, the workloads executing on the Cray EX may not exhibit the best possible performance, but the results show that our optimization can still provide performance improvements.

2) *SpMV*: The runtime speed-ups achieved on SpMV are very similar across all five systems, and significantly smaller than for IG. Unlike IG which issues uniformly random memory accesses, SpMV's access pattern depends on the non-zero pattern of the input sparse matrix. For our experiments, we evaluated a large variety of sparse matrices, some with highly

TABLE IV  
AVERAGE RUNTIME SPEED-UPS FROM ADAPTIVE PREFETCHING FOR THE  
SYSTEMS FROM TABLE II ON EACH WORKLOAD

| Workload       | FDR-56 | HDR-100 | HDR-200 | Cray XC | Cray EX |
|----------------|--------|---------|---------|---------|---------|
| IG             | 94     | 119     | 140     | 9       | 292     |
| SpMV           | 1.05   | 1.07    | 1.07    | 1.06    | 1.07    |
| PR             | 0.99   | 1.09    | 1.08    | 1.03    | 1.10    |
| SSSP           | 1.21   | 1.39    | 1.35    | 1.33    | 1.53    |
| <b>geomean</b> | 3.3    | 3.7     | 3.8     | 1.9     | 4.8     |

irregular sparsity patterns and some with more regular/dense structures (e.g., banded matrices). As a result, several matrices do not benefit from prefetching due to a lack of irregular memory accesses to exploit. However, because our optimization can detect such cases and pause prefetching, performance is not negatively impacted in most cases. Specifically, of the 1,012 SpMV experiments across the five systems, for 37 sparse matrices, and different number of locales, performance loss is avoided in 95% of the results. However, there are 60 cases where performance loss exceeds 10% due to an insufficient amount of remote communication for prefetching to exploit, which is the result of running on a small number of locales (i.e., four or less) and/or processing a small matrix (i.e., less than 500 million non-zeros). Additionally, the relatively small average speed-ups for SpMV in Table IV are largely due to the fact that 63% of the 1,012 SpMV experiments exhibit no noticeable gain or loss in performance (i.e., speed-up of 1x). If we exclude the 60 cases where performance losses are larger than 10%, as well as cases where performance is neither improved nor degraded, then the average speed-ups are 1.4x, 1.3x, 1.2x, 1.5x, and 1.3x for the FDR-56, HDR-100, HDR-200, Cray XC, and Cray EX systems, respectively. Beyond average speed-ups, prefetching can provide gains as large as 3.3x, 3.7x, 3.6x, 2.4x, and 2.8x on the FDR-56, HDR-100, HDR-200, Cray XC, and Cray EX systems, respectively.

3) *PR*: For PR, we evaluated three real-world graphs from Table III, namely arabic-2005, sk-2005, and MOLIERE\_2016, as well as the Graph500 graph s-28. The underlying kernel in PR is an SpMV-like operation, so most systems achieve similar speed-ups compared to their SpMV results. The exceptions are the FDR-56 and Cray XC systems, which achieve good speed-ups on the larger and more irregular graphs (MOLIERE\_2016 and s-28) but do not perform as well on the smaller and more structured graphs (arabic-2005 and sk-2005). As mentioned earlier, the FDR-56 system has less bandwidth and can suffer from too many in-flight prefetches, while the Cray XC's Aries interconnect already provides good performance for fine-grain communication. While prefetching can be applied to the PR kernel, a better solution is to use selective data replication (SDR) [9]. As described briefly in Section III-D3, SDR will create explicit copies of the remotely accessed data prior to the kernel (i.e., `forall` loop) and then redirect the remote accesses to the replicated data. PR is unique amongst the other workloads because it has an outer loop that executes the kernel multiple times. As a result, SDR can be applied because the

cost of the inspector/replication can likely be amortized over multiple executions of the kernel, leading to speed-ups as large as 36x.

4) *SSSP*: The SSSP workload is the most complicated in terms of what each iteration of the target `forall` loop performs. Specifically, the loop contains two irregular memory access patterns, where only one is a valid candidate for prefetching. The other access pattern involves writes, which can be optimized via small message aggregation and is left for future work. Additionally, the loop also exhibits branching/divergent behavior, which can lead to wasted prefetches (i.e., prefetches for data that will never be accessed). Despite these complexities, the prefetching optimization can provide fairly significant average speed-ups on each system when evaluating the Graph500 graphs s-26, s-27, and s-28.

#### F. Adaptive Heuristic Evaluation

The adaptive heuristic described in Section III-C and illustrated in Figure 1 will pause prefetching if too many unnecessary prefetches are detected, as well as resume prefetching if the cache miss rate is too high. Additionally, the heuristic will adjust the prefetch distance to ensure that prefetches are on time. In this section, we present an evaluation of the heuristic to demonstrate the advantages of adjusting the distance and pausing/resuming prefetches.

1) *Pausing/Resuming Prefetches*: To demonstrate the advantages of pausing/resuming prefetches, we evaluated SpMV on the HDR-200 system under three different conditions: (1) the heuristic only adjusts the prefetch distance (ADJUST-ONLY), (2) the heuristic adjusts the distance and will pause prefetches if deemed necessary but will never resume prefetches (ADJUST+PAUSE), and (3) the heuristic adjusts the distance, pauses prefetches, and will resume prefetches if necessary (ADJUST+PAUSE+RESUME). SpMV is a suitable workload for these experiments because of the large number of different sparse matrix data sets from Table III that we can evaluate, which can provide insight into when different components of the heuristic are helpful for performance and when they are not.

Figure 2 presents the runtime speed-ups achieved by each prefetching option relative to the no-prefetching baseline on 64 locales. To begin, ADJUST-ONLY results in an average performance loss of 25%, where only 8 of the 37 matrices exhibit runtime speed-ups. The performance loss is due to prefetching even when it is not likely to provide a benefit, such as when regular/dense structures in the sparse matrices are encountered. On the other hand, when the matrices are highly irregular, such as MOLIERE\_2016 and kron\_g500-logn21, ADJUST-ONLY does well since there is less of a need to pause prefetching.

When we enable the ability to pause prefetches (ADJUST+PAUSE), then average performance improves from 0.75x to 1x (i.e., neither a gain nor loss in performance). By allowing prefetching to be paused when too many unnecessary prefetches are encountered, we avoid most of the situations where we lost performance due to always prefetching.

However, for the MOLIERE\_2016 matrix, pausing prefetches results in a 2.8x loss in performance versus always prefetching. In this case, most of the tasks stopped prefetching due to encountering some dense/regular memory access patterns. By doing so, the remaining irregular memory access patterns are not prefetched.

Finally, when the ability to resume prefetches is enabled (ADJUST+PAUSE+RESUME), the average runtime speed-up is improved from 1x to 1.07x. In most cases, allowing tasks to resume prefetches when there are too many cache misses results in improved performance over ADJUST-ONLY and ADJUST+PAUSE. The MOLIERE\_2016 matrix is an interesting case, as it clearly shows the benefits of resuming prefetches by providing a 3.4x improvement over ADJUST+PAUSE and a 1.2x improvement over ADJUST-ONLY. However, there are still some matrices where ADJUST+PAUSE+RESUME does not outperform ADJUST-ONLY, namely GAP-kron, kron\_g500-logn21, and AGATHA\_2015. These three matrices exhibit the largest number of pause/resume cycles, which indicates large amounts of alternating regular/irregular portions of the memory access pattern. This presents a challenge for the heuristic, as excessive pausing and resuming complicates the ability of the heuristic to effectively adjust the prefetch distance.

2) *Benefits of Prefetch Distance Adjustment:* The results presented so far do not provide direct insight into the benefits of adjusting the prefetch distance over time. To show that, we monitored how many prefetches were issued with a given distance across all of the tasks for a given execution of PR and SSSP on the same Graph500 graph using 8 locales. The results are shown in Figure 3. There are three key observations from these results: (1) while PR and SSSP are operating on the same input graph, they exhibit different prefetch usage patterns; (2) the prefetch usage patterns vary across the five systems for the same workload and data set; and (3) there is no obvious preferred distance that can be used by all tasks. These observations underscore the importance of using an adaptive approach for the prefetch distance. To illustrate this further, we used the results in Figure 3 to determine the most commonly used distance across all tasks for PR and SSSP on the HDR-200 system. Using these distances, we ran PR and SSSP again but instead of adapting the distances we held them constant at the specified values. We found that the performance of the adaptive approach is 3% and 35% better for PR and SSSP, respectively, when compared to using a constant distance. The performance difference is much larger for SSSP because it is a more complex application and exhibits other remote communication besides the access pattern that is being prefetched, which can affect the timings of the prefetches. This can be seen in the heatmap for SSSP on the HDR-200 system in Figure 3, which exhibits a large variation in the prefetch distances used.

#### G. Comparison to Other PGAS Implementations

Our prefetching optimization and accompanying compiler support are implemented within the Chapel programming

language and compiler. Our results thus far have shown that the performance of irregular workloads in Chapel can be significantly improved by the adaptive prefetching optimization. However, there are other PGAS languages and libraries besides Chapel, such as OpenSHMEM, UPC, and the remote memory access features of MPI (MPI-RMA). To compare the performance of our optimization within Chapel to those PGAS languages/libraries, we implemented the IG and SpMV workloads in OpenSHMEM, UPC, and MPI-RMA, matching the Chapel implementation style in terms of data distributions and communication patterns. The goal of these evaluations is to emphasize the importance of caching remote data in PGAS programs, and using optimizations to make effective use of such caches.

On the HDR-200 system, we observe that the Chapel baseline (no-prefetching) of IG is on average 6.4x, 5.7x, and 1.6x slower than OpenSHMEM, UPC, and MPI-RMA, respectively. The large difference in performance is due to Chapel's fork-join execution model versus the SPMD model of the other languages/libraries. Specifically, GASNet over Infiniband serializes communication injection onto the network within a process, which can hinder performance for a workload like IG that issues many small messages from multiple threads. A Chapel "process" is inherently multi-threaded while the other PGAS implementations operate in a one-process-per-core fashion, which does not lead to such communication contention. However, when we apply our prefetching optimization to the Chapel implementation, we achieve average speed-ups of 22x, 24x, and 88x over OpenSHMEM, UPC, and MPI-RMA, respectively. Unlike Chapel, the other PGAS implementations do not have a remote cache, so remote data cannot be prefetched without significant effort (e.g., implementing a remote cache).

For SpMV, we found that Chapel without prefetching is on average 16x, 14x, and 201x faster than OpenSHMEM, UPC, and MPI-RMA, respectively. The large gains are due to the other PGAS implementations not having a remote cache, which severely hinders performance for the sparse matrices that exhibit some amount of dense/regular memory access patterns. When prefetching is enabled for Chapel, we achieve additional improvements as large as 3.4x, 3.4x, and 44x over OpenSHMEM, UPC, and MPI-RMA, respectively, with an average improvement of 1.06x.

#### V. CONCLUSIONS AND FUTURE WORK

While the Partitioned Global Address Space (PGAS) model offers productivity benefits for writing irregular applications, it naturally leads programmers to produce code that requires fine-grain communication. Optimizations that address fine-grain communication performance, such as prefetching, can be applied by-hand but at the loss of user productivity. In this work we have presented an adaptive prefetching optimization for PGAS programs that can leverage information at runtime to adjust prefetching behavior. Furthermore, we developed compiler support for the optimization so it can be automatically applied with no user intervention. By adjusting prefetching behavior at runtime, our optimization can be used

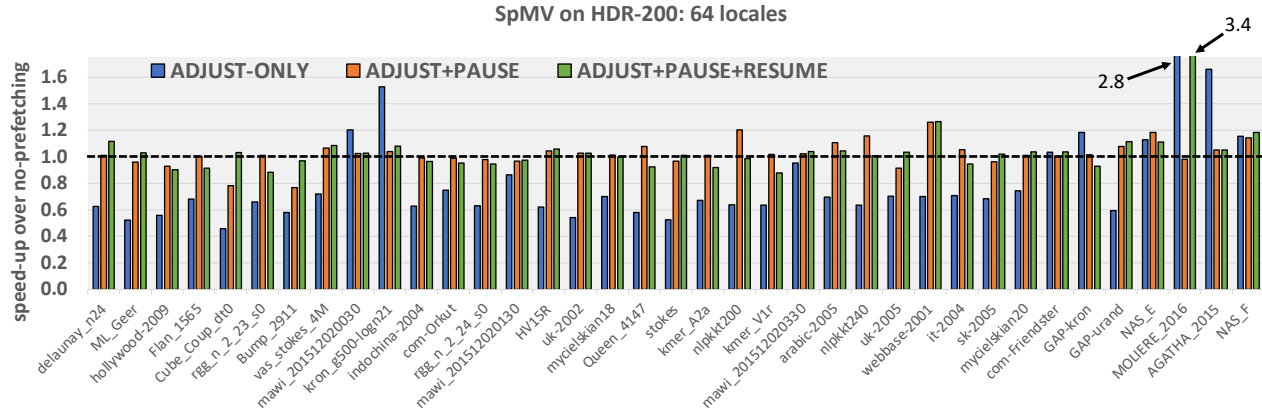


Fig. 2. Runtime speed-ups achieved via adaptive prefetching versus no-prefetching for SpMV on the HDR-200 system using 64 locales. Each bar corresponds to a version of the prefetch heuristic that performs a combination of adjusting the prefetch distance, pausing prefetches, and/or resuming prefetches. Higher bars correspond to better performance, where bars below the dashed line represent performance loss w.r.t. no-prefetching and bars above the dashed line represent performance gains. For the MOLIÈRE\_2016 matrix, speed-ups for ADJUST-ONLY and ADJUST+PAUSE+RESUME exceed the maximum y-axis values but are indicated directly on the chart.

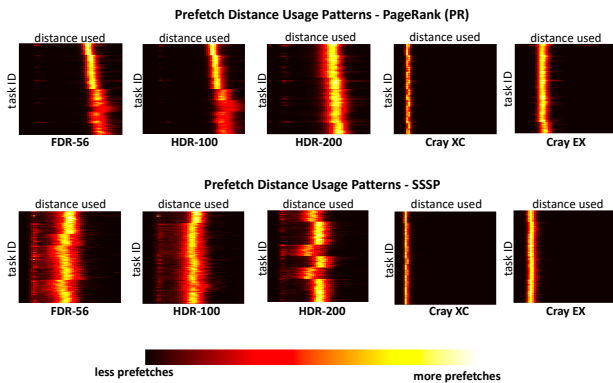


Fig. 3. Heatmaps that show each task's prefetch distance usage pattern when running PR and SSSP on the same Graph500 graph with 8 locales on the systems from Table II. The horizontal rows in a map represent individual tasks and the vertical columns represent prefetch distances used, which range from 1 to 50. Values in a map represent how many prefetches were issued using a given distance, where darker colors correspond to fewer prefetches and lighter colors correspond to more prefetches.

on different systems, workloads, and data sets without static tuning. Through this optimization, we demonstrated performance improvements as large as 377x when compared to not prefetching. Since the optimization is applied automatically by the compiler, programmers can achieve large performance improvements without negatively impacting their productivity. For future work, we plan to improve the optimization and explore machine learning techniques to select values for the heuristic parameters.

## REFERENCES

- [1] M. De Wael, S. Marr, B. De Fraine, T. Van Cutsem, and W. De Meuter, "Partitioned global address space languages," *ACM Computing Surveys (CSUR)*, vol. 47, no. 4, pp. 1–27, 2015.
- [2] B. L. Chamberlain, E. Ronaghan, B. Albrecht, L. Duncan, M. Ferguson, B. Harshbarger, D. Iten, D. Keaton, V. Litvinov, P. Sahabu *et al.*, "Chapel comes of age: Making scalable programming productive," 2018.
- [3] T. El-Ghazawi, W. Carlson, T. Sterling, and K. Yelick, *UPC: Distributed Shared Memory Programming*, ser. Wiley Series on Parallel and Distributed Computing. NJ, USA: John Wiley & Sons, 2005, vol. 40.
- [4] B. Chapman, T. Curtis, S. Pophale, S. Poole, J. Kuehn, C. Koelbel, and L. Smith, "Introducing OpenSHMEM: SHMEM for the PGAS community," in *Proceedings of the Fourth Conference on Partitioned Global Address Space Programming Model*, ser. PGAS '10. New York, NY, USA: Association for Computing Machinery, 2010. [Online]. Available: <https://doi.org/10.1145/2020373.2020375>
- [5] D. Bader, P. Kogge, A. Lumsdaine, R. Murphy, T. Hoefer, and A. Korzh, "Graph500 benchmark," <https://graph500.org/>, 2022.
- [6] J. Dongarra, M. A. Heroux, and P. Luszczek, "High-performance conjugate-gradient benchmark: A new metric for ranking high-performance computing systems," *The International Journal of High Performance Computing Applications*, vol. 30, no. 1, pp. 3–10, 2016.
- [7] W.-Y. Chen, C. Iancu, and K. Yelick, "Communication optimizations for fine-grained upc applications," in *14th International Conference on Parallel Architectures and Compilation Techniques (PACT'05)*, 2005, pp. 267–278.
- [8] T. B. Rolinger, J. Craft, C. D. Krieger, and A. Sussman, "Towards high productivity and performance for irregular applications in chapel," in *2021 IEEE/ACM Parallel Applications Workshop, Alternatives To MPI (PAW-ATM)*. St. Louis, MO, USA: IEEE, 2021, pp. 1–11.
- [9] T. B. Rolinger, C. D. Krieger, and A. Sussman, "Compiler optimization for irregular memory access patterns in PGAS programs," in *Languages and Compilers for Parallel Computing: 35th International Workshop, LCPC 2022, Chicago, IL, USA, October 12–14, 2022*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 3–21. [Online]. Available: [https://doi.org/10.1007/978-3-031-31445-2\\_1](https://doi.org/10.1007/978-3-031-31445-2_1)
- [10] M. Grossman, H. Pritchard, Z. Budimlić, and V. Sarkar, "Graph500 on OpenSHMEM: Using a practical survey of past work to motivate novel algorithmic developments," in *Proceedings of the Second Annual PGAS Applications Workshop*, 2017, pp. 1–8.
- [11] A. J. Smith, "Cache memories," *ACM Computing Surveys (CSUR)*, vol. 14, no. 3, pp. 473–530, 1982.
- [12] F. Dahlgren and P. Stenstrom, "Evaluation of hardware-based stride and sequential prefetching in shared-memory multiprocessors," *IEEE Transactions on Parallel and Distributed Systems*, vol. 7, no. 4, pp. 385–398, 1996.
- [13] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *2007 IEEE 13th International Symposium on High Performance Computer Architecture*. IEEE, 2007, pp. 63–74.
- [14] N. Talati, K. May, A. Behroozi, Y. Yang, K. Kaszyk, C. Vasiladiotis, T. Verma, L. Li, B. Nguyen, J. Sun *et al.*, "Prodigy: Improving the

- memory latency of data-indirect irregular workloads using hardware-software co-design,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 654–667.
- [15] S. Ainsworth and T. M. Jones, “Software prefetching for indirect memory accesses,” in *2017 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. Austin, TX, USA: IEEE, 2017, pp. 305–317.
  - [16] D. Callahan, K. Kennedy, and A. Porterfield, “Software prefetching,” *ACM SIGARCH Computer Architecture News*, vol. 19, no. 2, pp. 40–52, 1991.
  - [17] A.-H. Badawy, A. Aggarwal, D. Yeung, and C.-W. Tseng, “The efficacy of software prefetching and locality optimizations on future memory systems,” *Journal of Instruction-Level Parallelism*, vol. 6, no. 7, 2004.
  - [18] M. P. Ferguson and D. Buettner, “Caching puts and gets in a PGAS language runtime,” in *2015 9th International Conference on Partitioned Global Address Space Programming Models*. Washington, DC, USA: IEEE, 2015, pp. 13–24.
  - [19] R. Bianchini, R. Pinto, and C. L. Amorim, “Data prefetching for software DSMs,” in *Proceedings of the 12th international conference on Supercomputing*, 1998, pp. 385–392.
  - [20] H. Liu and W. Hu, “A comparison of two strategies of dynamic data prefetching in software DSM,” in *Proceedings 15th International Parallel and Distributed Processing Symposium. IPDPS 2001*. IEEE, 2001, pp. 6–pp.
  - [21] T. C. Mowry, C. Q. Chan, and A. K. Lo, “Comparative evaluation of latency tolerance techniques for software distributed shared memory,” in *Proceedings 1998 Fourth International Symposium on High-Performance Computer Architecture*. IEEE, 1998, pp. 300–311.
  - [22] T. Mowry and A. Gupta, “Tolerating latency through software-controlled prefetching in shared-memory multiprocessors,” *Journal of Parallel and Distributed Computing*, vol. 12, no. 2, pp. 87–106, 1991.
  - [23] A.-C. Lai and C.-L. Lei, “Data prefetching for distributed shared memory systems,” in *Proceedings of HICSS-29: 29th Hawaii International Conference on System Sciences*, vol. 1. IEEE, 1996, pp. 102–110.
  - [24] E. Kayraklioglu, M. P. Ferguson, and T. El-Ghazawi, “LAPPS: Locality-aware productive prefetching support for PGAS,” *ACM Trans. Archit. Code Optim.*, vol. 15, no. 3, Aug. 2018. [Online]. Available: <https://doi.org/10.1145/3233299>
  - [25] W. Heirman, K. D. Bois, Y. Vandriessche, S. Eyerman, and I. Hur, “Near-side prefetch throttling: Adaptive prefetching for high-performance many-core processors,” in *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, ser. PACT ’18. New York, NY, USA: Association for Computing Machinery, 2018. [Online]. Available: <https://doi.org/10.1145/3243176.3243181>
  - [26] R. Saavedra and D. Park, “Improving the effectiveness of software prefetching with adaptive executions,” in *Proceedings of the 1996 Conference on Parallel Architectures and Compilation Techniques*. Boston, MA, USA: IEEE, 1996, pp. 68–78.
  - [27] V. Jiménez, R. Gioiosa, F. J. Cazorla, A. Buyuktosunoglu, P. Bose, and F. P. O’Connell, “Making data prefetch smarter: Adaptive prefetching on power7,” in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques*, 2012, pp. 137–146.
  - [28] E. Ebrahimi, O. Mutlu, and Y. N. Patt, “Techniques for bandwidth-efficient prefetching of linked data structures in hybrid prefetching systems,” in *2009 IEEE 15th International Symposium on High Performance Computer Architecture*. IEEE, 2009, pp. 7–17.
  - [29] K. B. Wheeler, R. C. Murphy, and D. Thain, “Qthreads: An API for programming with millions of lightweight threads,” in *IEEE International Parallel and Distributed Processing Symposium*. Miami, FL, USA: IEEE, 2008, pp. 1–8.
  - [30] J. H. Saltz, R. Mirchandaney, and K. Crowley, “Run-time parallelization and scheduling of loops,” *IEEE Transactions on Computers*, vol. 40, no. 5, pp. 603–612, 1991.
  - [31] R. Das, M. Uysal, J. Saltz, and Y.-S. Hwang, “Communication optimizations for irregular scientific computations on distributed memory architectures,” *Journal of Parallel and Distributed Computing*, vol. 22, no. 3, pp. 462–478, 1994.
  - [32] M. M. Strout, L. Carter, and J. Ferrante, “Compile-time composition of run-time data and iteration reorderings,” in *Proceedings of the ACM SIGPLAN 2003 Conference on Programming language Design and Implementation*. San Diego, CA, USA: ACM, 2003, pp. 91–102.
  - [33] D. Bonachea and P. Hargrove, “GASNet specification, v1. 8.1,” Lawrence Berkeley National Lab.(LBNL), Berkeley, CA (United States), Tech. Rep., 2017.
  - [34] U. Meyer and P. Sanders, “ $\delta$ -stepping: a parallelizable shortest path algorithm,” *Journal of Algorithms*, vol. 49, no. 1, pp. 114–152, 2003.
  - [35] T. A. Davis and Y. Hu, “The University of Florida Sparse Matrix Collection,” *ACM Trans. Math. Softw.*, vol. 38, no. 1, pp. 1:1–1:25, Dec. 2011.
  - [36] D. Bailey, T. Harris, W. Saphir, R. Van Der Wijngaart, A. Woo, and M. Yarrow, “The NAS parallel benchmarks 2.0,” Technical Report NAS-95-020, NASA Ames Research Center, Tech. Rep., 1995.